PROBLEM SOLVING USING JAVA

PRACTICE EXERCISE

Detailed Case Study / Scenario-Based Questions | CO1–CO6

Maximum Marks: 90 **Instructions:** Answer all questions. Write complete Java programs with suitable classes and methods, accept input where required, validate data, and display meaningful output. For the algorithm question, provide both the algorithm and a clearly labelled flowchart.

CO1 – Algorithm and Flowchart

1. A college has introduced a campus shuttle service to help students travel between the main gate, academic blocks, library, hostels, and sports complex. The fare is ■10 per kilometre for the first 5 kilometres and ■7 for every kilometre beyond 5 kilometres; when the total distance is more than 20 kilometres, the student receives a 5% concession on the calculated fare. Design an algorithm that accepts the distance travelled, validates the input, calculates the fare, applies the concession when eligible, and displays the distance, base fare, discount amount, and final fare. Draw a flowchart showing the input, validation decision, fare calculation, discount decision, and output. Include a response for zero or negative distance.

CO2 – Pattern Programming

2. Write a Java program to print the following number pattern. Read the number of rows *n* from the user and use nested loops to generate the pattern. The program must work for any positive value of *n*; do not hard-code the rows. Validate the input before printing.

Pattern to be displayed:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Example shown for n = 5. The outer loop controls rows, and the inner loop prints values from 1 up to the current row number.

CO3 – Two-Dimensional Arrays (Matrix Operations)

3. A college examination cell wants to analyse marks stored in matrix form for academic reporting. Write a Java program that reads two matrices of order **3 × 3** from the user and stores them in two-dimensional arrays. Display both matrices in matrix format, calculate and display their matrix addition and subtraction, and calculate the transpose of the first matrix. If possible, also calculate the multiplication of the two matrices using the standard row-by-column rule. Use nested loops for input, processing, and output, and ensure the resulting matrices are displayed clearly with rows and columns aligned.

CO4 – Object-Oriented Programming and Exception Handling

4.a. A college sports centre is replacing its paper registration book with a Java application. Each registration stores a student's registration number, name, selected sport, and year of study. Create a **SportsStudent** class with suitable data members and a **displayDetails()** method. In a separate **SportsStudentMain** class, read details for at least two students, create separate objects, assign their data, and invoke the display method for each object to show that each object maintains independent information.

4.b. A college transport department offers a basic bus pass and a premium pass with additional benefits. Both share pass ID, student name, and base fee, while the premium pass includes a facility charge. Create a parent class **BusPass** with common data and a **displayPassDetails()** method. Extend it with **PremiumBusPass**, add the facility charge, and implement **calculateTotalFee()**. Read the details, create suitable objects, invoke the methods, and display pass information and the total payable amount, demonstrating inheritance and reuse.

4.c. A hostel administration system stores room numbers and monthly fees and must prevent invalid values from being assigned directly. Create a **HostelRoom** class with private fields **roomNumber** and **monthlyFee**, plus public getters and setters. Validate that the room number and monthly fee are positive. Read values, assign them through setters, retrieve them through getters, and display accepted values. If input is invalid, display a meaningful message and do not silently accept it. Demonstrate encapsulation and controlled access to private data.

4.d. A college laboratory does not allow students with overdue equipment to borrow additional items. Create a user-defined exception named **OverdueItemException**. Read a student's name and overdue-item count. If the count is

greater than zero, throw the exception with a meaningful message; otherwise, confirm that borrowing is approved. Handle the exception using **try-catch** without abruptly terminating the program, and reject a negative overdue-item count.

CO5 – Abstraction, Strings, Files, and Sorting

5.a. A college cafeteria provides standard and premium meal plans. Create an abstract class **MealPlan** with **studentName**, **numberOfDays**, and an abstract **calculateFee()** method. Create **StandardMealPlan** charging ■180 per day and **PremiumMealPlan** charging ■250 per day plus a fixed service fee of ■500. Read the student's name, duration, and plan, create the appropriate subclass object, and display the student's details, daily charge, applicable service fee, and total amount.

5.b. Use the following sample input to demonstrate String processing and mutable string classes. Clean the participant name using String methods, build and modify a message using StringBuilder, append a status using StringBuffer, and split the event list using StringTokenizer. Apply the StringBuilder methods listed below to a message based on the cleaned participant name and display intermediate results.

Sample input:
Participant name: **" arun kumar "**
Event list: **"Coding,Robotics,Project Expo"**
Registration status: **"Registration Pending"**

StringBuilder methods to apply:

```
1. append(text)              - add text at the end
2. insert(index, text)       - insert text at an index
3. replace(start, end, text) - replace a range
4. delete(start, end)        - delete a range
5. deleteCharAt(index)       - delete one character
6. setCharAt(index, ch)      - replace one character
7. reverse()                 - reverse the sequence
8. capacity()                - get current capacity
9. length()                  - get current length
10. charAt(index)            - get a character
11. substring(start)         - get a substring
12. substring(start, end)    - get a substring range
13. indexOf(text)            - find first occurrence
14. lastIndexOf(text)        - find last occurrence
15. ensureCapacity(minimum)  - request minimum capacity
16. setLength(newLength)     - change logical length
17. trimToSize()             - reduce excess capacity
```

Requirements: Use String methods such as **trim()**, **equalsIgnoreCase()**, **contains()**, and **toUpperCase()** or **toLowerCase()**. Create a StringBuilder from the cleaned name and apply the listed methods where appropriate, displaying intermediate results. Use valid indices for range- and character-based methods because invalid indices cause exceptions. Demonstrate **StringBuffer** by appending the registration status and **StringTokenizer** with a comma delimiter to print each event separately. Note that inspection methods such as **capacity()**, **length()**, **charAt()**, **substring()**, **indexOf()**, and **lastIndexOf()** do not modify the builder.

5.c. The placement training cell wants to store workshop registrations in a text file instead of handwritten registers. Read a participant's name, registration number, department, and workshop title, write the details in a clear format to **workshop.txt**, then open the same file for reading and display its contents on the console. Use suitable file reader/writer classes, handle file-related exceptions, and close resources properly, preferably with try-with-resources. Demonstrate both writing and reading.

5.d. A college placement cell stores student registration number, name, department, and aptitude score. It needs to sort records by aptitude score and alphabetically by name. Create a **PlacementStudent** class implementing **Comparable<PlacementStudent>** to define natural ordering by aptitude score. Then use a **Comparator<PlacementStudent>** or comparator expression to sort by student name. Store several objects in a collection, display records before sorting, sort and display using Comparable, and then sort and display using Comparator. Handle equal scores or names consistently.

CO6 – Collections, Lambda Expressions, and Generics

6.a. The placement office needs a flexible list of students registered for campus interviews. Create a **PlacementStudent** class with registration number, name, department, and placement status, and store objects in an **ArrayList<PlacementStudent>**. Allow the user to add records, display all records, search by registration number, and remove a selected student. If the list is empty or a student is not found, display an appropriate message. Use loops to traverse the collection and demonstrate storing user-defined objects dynamically.

6.b. A college library lends project equipment such as microcontrollers, sensors, and networking kits. To handle requests in arrival order, create an **EquipmentRequest** class with request ID, student name, equipment name, and purpose, and store objects in a **LinkedList<EquipmentRequest>**. Write a program to add requests at the end, display pending requests, view the first request, and remove it when equipment is issued. If the list is empty, display a suitable message instead of accessing a missing element. Demonstrate queue-style processing.

6.c. A college examination section needs to calculate grace-adjusted marks. Add 5 grace marks to the original mark, but ensure the final mark does not exceed 100. Define a functional interface named **MarkProcessor** with exactly one abstract method that accepts an integer and returns the processed mark. Use a lambda expression to implement the method, read a mark, apply the lambda, and display original and adjusted marks. Validate that the original mark is between 0 and 100.

6.d. A college administration system stores different types of information, including student names, semester numbers, and grade-point averages. Create a generic class named **DataBox<T>** with a private field of type T, a constructor, getter, and setter. Create objects such as **DataBox<String>**, **DataBox<Integer>**, and **DataBox<Double>**, assign and update values, and display them. Explain how the type parameter supports reuse and compile-time type safety.

End of Question Paper

PROBLEM SOLVING USING JAVA

PRACTICE EXERCISE

Maximum Marks: 90 **Instructions:** Answer all questions. Write complete Java programs with suitable classes and methods, accept input where required, validate data, and display meaningful output. For the algorithm question, provide both the algorithm and a clearly labelled flowchart.

CO1 – Algorithm and Flowchart

1. A college canteen is introducing a digital billing system to help students calculate the cost of their purchases before paying. The cashier enters the total number of items purchased and the total bill amount. If the bill amount is greater than ■1,000, the student receives a 10% discount; otherwise, no discount is applied. Design an algorithm that reads the bill amount, validates that it is positive, calculates the applicable discount and final payable amount, and displays the original bill, discount, and final amount. Draw a flowchart that clearly represents input, validation, the discount decision, calculation, and output. Include a suitable message for invalid input.

CO2 – Arithmetic Operations Using switch Statement

2. Write a Java program that reads two numbers and an arithmetic operator from the user. Use a **switch** statement to perform the selected operation: addition (+), subtraction (-), multiplication (*), division (/), or remainder (%). Display the calculation and result in a clear format. If the user enters an unsupported operator, display an appropriate error message. Before performing division or remainder, check whether the second number is zero and display a message instead of attempting an invalid operation. The program must use a switch statement for operation selection rather than an if-else ladder.

CO3 – One-Dimensional Array

3. A college sports department records the scores achieved by participants in a selection trial. The coach wants to identify the highest score to determine the leading participant. Write a Java program that reads the number of participants, accepts each participant's score into a one-dimensional integer array, and finds the maximum element without sorting the array or using built-in maximum functions. Display all entered scores and the maximum score. Validate that the number of participants is positive and that the scores are within an appropriate range. Use a loop to initialise the maximum from the first array element and compare it with the remaining elements.

CO4 – Object-Oriented Programming and Exception Handling

4.a. A college innovation hub wants to maintain details of students who register for project demonstrations. Each record must include the student's registration number, full name, project title, and team size. Create a class named **ProjectParticipant** with suitable data members and a method **displayDetails()** to print the complete record. In a separate **ProjectParticipantMain** class, read the details of at least two participants, create separate objects, assign their data, and call the display method for each object. The output should demonstrate that each object stores and displays its own participant details.

4.b. A college transport office manages ordinary and express shuttle passes. Both pass types share a pass ID, student name, and base fare, while an express pass includes an additional express-service charge. Create a parent class named **ShuttlePass** containing the common fields and a method **displayPassDetails()**. Create a child class named **ExpressShuttlePass** that extends the parent class, adds the express charge, and defines **calculateTotalFare()** to return the base fare plus the additional charge. Read the required details, create the objects, invoke the methods, and display the pass details and final fare. Demonstrate how the child class inherits common properties and behaviour from the parent class.

4.c. A college laboratory issues equipment kits to students and needs to maintain the kit number and refundable deposit assigned to each student. To avoid incorrect records, create a class named **EquipmentKit** with private data members **kitNumber** and **depositAmount**. Provide public getter and setter methods for both fields. The setter methods must validate that the kit number is positive and the deposit amount is greater than zero. Write a main program to read the values, assign them using setters, retrieve them using getters, and display the accepted information. If an invalid value is supplied, display a meaningful message and ensure that invalid data is not silently accepted. The program should demonstrate encapsulation and controlled access to private fields.

4.d. A college project laboratory allows students to reserve expensive devices, but a student who has damaged or failed to return a previously borrowed device cannot make another reservation. Create a user-defined exception named **DeviceNotReturnedException**. Read the student's name and a value indicating whether a previous device is still unreturned. If a device is pending, throw the exception with a meaningful message; otherwise, confirm that the reservation is accepted. Handle the exception using **try-catch** so the application continues normally, and validate the

input so that only the expected values are accepted.

CO5 – Abstraction, Strings, Files, and Sorting

5.a. A college fitness centre offers two membership plans. A basic membership charges ■300 per month, while a premium membership charges ■500 per month and includes a one-time registration fee of ■400. Create an abstract class named **FitnessMembership** with data members **memberName** and **numberOfMonths**, and declare an abstract method **calculateFee()**. Create **BasicMembership** and **PremiumMembership** subclasses that implement the fee calculation according to their respective rules. Read the member's name, membership duration, and selected plan, create the appropriate object, and display the member details, monthly rate, any additional fee, and total amount payable. Demonstrate how abstract classes define a common structure while subclasses provide their own implementation.

5.b. Use the following sample input to demonstrate String processing and mutable string classes. Clean the participant name using String methods, build and modify a message using StringBuilder, append a status using StringBuffer, and split the activity list using StringTokenizer. Apply the StringBuilder methods listed below to a message based on the cleaned name and display intermediate results.

Sample input:
Participant name: **" meena devi "**
Activity list: **"Coding,Quiz,Robotics Expo"**
Registration status: **"Payment Pending"**

StringBuilder methods to apply:

```
1. append(text)              - add text at the end
2. insert(index, text)       - insert text at an index
3. replace(start, end, text) - replace a range
4. delete(start, end)        - delete a range
5. deleteCharAt(index)       - delete one character
6. setCharAt(index, ch)      - replace one character
7. reverse()                 - reverse the sequence
8. capacity()                - get current capacity
9. length()                  - get current length
10. charAt(index)            - get a character
11. substring(start)         - get a substring
12. substring(start, end)    - get a substring range
13. indexOf(text)            - find first occurrence
14. lastIndexOf(text)        - find last occurrence
15. ensureCapacity(minimum)  - request minimum capacity
16. setLength(newLength)     - change logical length
17. trimToSize()             - reduce excess capacity
```

Requirements: Use String methods such as **trim()**, **equalsIgnoreCase()**, **contains()**, and **toUpperCase()** or **toLowerCase()**. Create a StringBuilder from the cleaned name and apply the listed methods where appropriate, displaying intermediate results. Use valid indices for range- and character-based methods because invalid indices cause exceptions. Use **StringBuffer** to append the registration status and **StringTokenizer** with a comma delimiter to print each activity separately. Note that inspection methods such as **capacity()**, **length()**, **charAt()**, **substring()**, **indexOf()**, and **lastIndexOf()** do not modify the builder.

5.c. A college career guidance cell wants to maintain a permanent record of students who attend resume-writing workshops. Read a student's name, registration number, department, and workshop date, write the details to a text file named **career_workshop.txt**, and then read the file and display the saved information on the console. Use suitable Java file-handling classes, handle file-related exceptions, and close resources properly, preferably using try-with-resources. The program should confirm that the details have been stored and retrieved successfully.

5.d. A college scholarship committee maintains records containing student ID, name, department, and cumulative grade-point average. The committee wants to rank students by grade-point average and also arrange the records alphabetically for document preparation. Create a **ScholarshipStudent** class implementing **Comparable<ScholarshipStudent>** to define natural ordering by grade-point average. Then use a **Comparator<ScholarshipStudent>** or comparator expression to sort the same objects by student name. Store several objects in a collection, display them before sorting, and display the results after each sorting operation. Ensure the comparison logic handles equal values consistently.

CO6 – Collections, Lambda Expressions, and Generics

6.a. A college cultural committee needs to maintain a dynamic list of students who register for events. Each student record contains registration number, name, department, and selected event. Create a **CulturalParticipant** class and store its objects in an **ArrayList<CulturalParticipant>**. Write a Java program that allows the user to add participant records, display all participants, search for a participant by registration number, and remove a selected participant. If the list is empty or the specified registration number does not exist, display a suitable message. Use loops to traverse the ArrayList and demonstrate how a collection can dynamically store user-defined objects.

6.b. A college IT help desk receives requests from students who need assistance with software installation, network access, or laboratory systems. To process requests in arrival order, create a **HelpDeskRequest** class with ticket number, student name, issue description, and priority. Store request objects in a **LinkedList<HelpDeskRequest>**. Write a program to add a new request at the end, display all pending requests, view the first request, and remove it when the issue is resolved. If no requests are pending, display an appropriate message. Demonstrate how LinkedList operations can be used to maintain and process a queue of objects.

6.c. A college examination office needs a reusable rule to determine whether a student has passed a programming test. Define a functional interface named **ResultChecker** with exactly one abstract method that accepts a mark and returns a boolean value. Use a lambda expression to implement the rule that a student passes when the mark is at least 50. Read a student's mark, apply the lambda, and display the mark and pass/fail result. Validate that the mark is between 0 and 100. The program should demonstrate how a lambda expression provides the implementation of a functional interface without creating a separate implementation class.

6.d. A college administration application needs to store values of different types, such as a student's name, semester number, and CGPA, without creating separate container classes for each type. Create a generic class named **RecordBox<T>** with a private variable of type T, a constructor, and getter and setter methods. In the main method, create **RecordBox<String>**, **RecordBox<Integer>**, and **RecordBox<Double>** objects, store appropriate values, update at least one value through its setter, and display the values through getters. Explain how generics improve code reuse and type safety.

End of Question Paper

PROBLEM SOLVING USING JAVA

PRACTICE EXERCISE

Maximum Marks: 90

Instructions: Answer all questions. Write readable Java programs with appropriate variable names, input validation, and comments wherever necessary. Draw neat and properly labelled flowcharts where required.

CO1 – Algorithm and Flowchart (5 Marks)

1. A campus cafeteria wants to automate the preparation of a customer bill. The system should accept the quantity and price of each item, calculate the subtotal, apply a 10% discount when the subtotal is greater than ■1,000, and finally display the discount and amount payable. Write a step-by-step algorithm for this process and draw a flowchart showing the sequence, decision, calculations, and output.

CO2 – Armstrong Number (5 Marks)

2. Write a Java program to read a non-negative integer and determine whether it is an Armstrong number. A number is an Armstrong number when the sum of each digit raised to the power of the number of digits is equal to the original number. For example, 153 is an Armstrong number because $1^3 + 5^3 + 3^3 = 153$. Your program must count the digits, process each digit using loops, calculate the required sum, and display a suitable result. Handle zero and single-digit inputs correctly.

CO3 – One-Dimensional Array and Linear Search (5 Marks)

3. A department stores the registration numbers of students who have enrolled for a coding workshop in a one-dimensional integer array. Write a Java program to read the number of students, store their registration numbers in an array, and then accept a registration number to search for. Implement linear search by comparing the target with array elements one by one. If the number is found, display its position; otherwise, display a message stating that the registration number is not present. Do not use predefined searching methods.

CO4 – Classes, Objects, Inheritance, Getters/Setters and User-Defined Exception (25 Marks)

4. A university project exhibition portal maintains details of student teams, their projects, and the equipment issued to them. Design a class named ProjectTeam with suitable private data members such as team ID, team name, project title, and number of members. Create objects, initialize the details through a constructor, and provide methods to display the team information. Extend the design using inheritance by creating a ResearchProjectTeam class that adds a research domain and overrides a display method to show both inherited and specialized details. Use getters and setters to access and update private fields, and validate values such as a non-empty team name and a positive number of members. Finally, define a user-defined exception named InvalidTeamDetailsException and throw it when invalid team information is entered; catch the exception in the main method and display an appropriate error message. Demonstrate the complete design with suitable objects and sample input.

CO5 – Abstract Classes, Strings, Files, Comparable and Comparator (25 Marks)

5(a). A university fitness centre offers regular and premium membership plans. Create an abstract class FitnessMembership with common data members and an abstract method to calculate the membership fee. Implement the method in RegularMembership and PremiumMembership subclasses using different fee rules. Create objects for both plans and display their membership details and calculated fees.

5(b). A student activity application receives and processes the following sample input: **Participant name:** " meena devi "; **Activities:** "Coding,Quiz,Robotics Expo"; **Status:** "Payment Pending". Write a Java program demonstrating useful methods from String, StringBuilder, StringBuffer, and StringTokenizer. Use String methods to trim extra spaces, change letter case, check whether text contains a word, compare strings, and locate text. Use StringTokenizer with a comma delimiter to separate and display the activities. Use

StringBuilder to modify the participant name and status, and StringBuffer to append additional text. Demonstrate and explain suitable StringBuilder methods, including append(text), insert(index,text), replace(start,end,text), delete(start,end), deleteCharAt(index), setCharAt(index,ch), reverse(), capacity(), length(), charAt(index), substring(start), substring(start,end), indexOf(text), lastIndexOf(text), ensureCapacity(minimum), setLength(newLength), and trimToSize().

5(c). A workshop coordinator stores the length and breadth of a rectangle in a text file named **rectangle.txt**. Write a Java program that reads the length and breadth from the file, calculates the area using the formula area = length × breadth, and writes the input values and calculated area back into the same file. Ensure that file-reading and file-writing resources are properly closed, and handle possible file-related or invalid-input exceptions.

5(d). A scholarship portal stores student records that must be sorted in different ways. Create a Student class with student ID, name, and GPA. Implement Comparable to sort students by GPA in descending order. Then create a Comparator to sort the same student objects alphabetically by name. Demonstrate both sorting orders using a collection of student objects and display the results.

CO6 – LinkedList, HashSet, Functional Interface and Generics (25 Marks)

6(a). A campus help desk manages service requests in the order in which they arrive. Create a HelpDeskRequest class containing request ID, student name, and issue description. Store request objects in a LinkedList and demonstrate adding requests, displaying all requests, retrieving the first request, removing a completed request, and checking whether the list is empty. Explain how the LinkedList supports the help-desk workflow.

6(b). A university event registration system must avoid storing duplicate participant records. Create a Participant class with participant ID and name, then store Participant objects in a HashSet. Demonstrate adding objects, checking membership, removing an object, and displaying the set. Override equals() and hashCode() so that two Participant objects with the same participant ID are treated as duplicates. Test the program by attempting to add two objects with the same ID but different names.

6(c). Create a functional interface named ResultChecker with one abstract method that accepts a student's mark and returns whether the student has passed. Implement the interface using a lambda expression with a pass mark of 50. Read or assign sample marks for multiple students and use the lambda to display Pass or Fail for each student. Briefly demonstrate why the interface qualifies as a functional interface.

6(d). Develop a generic class named RecordBox that stores one value of type T and provides methods to set and retrieve the stored value. Demonstrate the same generic class with a String, an Integer, and a Double. Show that the class can work with different data types without writing separate classes for each type, and explain how the type parameter helps provide compile-time type safety.